Министерство образования Республики Беларусь

Учреждение образования «Белорусский государственный университет информатики и радиоэлектроники»

Факультет компьютерных систем и сетей
Кафедра Информатики
Дисциплина «Программирование мобильных приложений»

ОТЧЁТ
к лабораторной работе №3
на тему
«КАЛЬКУЛЯТОР»
БГУИР 6-05-0612-02 23

Выполнил студент группы 353503
СЕБЕЛЕВ Дмитрий Юрьевич

_____
(дата, подпись студента)

Проверил ассистент каф. информатики
БУРАКОВ Дмитрий Игоревич

_____
(дата, подпись преподавателя)

Минск 2026

СОДЕРЖАНИЕ

1 Постановка задачи 3
2 Выполнение работы 4
2.1 Архитектура и состав приложения 4
2.2 Режимы работы и интерфейс 4
2.3 Математическая логика 6
2.4 Конфигурации сборки (Flavors) 7
Вывод 8
Приложение А (обязательное) Листинг программного кода 9

1 ПОСТАНОВКА ЗАДАЧИ

Целью лабораторной работы являлась разработка мобильного приложения «Калькулятор» для операционной системы *Android*. Приложение должно поддерживать базовые и инженерные вычисления, адаптироваться к ориентации устройства и предоставлять различные конфигурации сборки.

Функциональные требования к приложению включают следующие возможности:

– Реализация базового режима вычислений: сложение, вычитание, умножение, деление и вычисление процентов.

– Реализация инженерного режима вычислений: факториал, возведение в степень, поддержка скобок и извлечение квадратного корня.

– Автоматическое переключение в инженерный режим при ландшафтной ориентации устройства.

– Возможность ручного переключения режимов в портретной ориентации с помощью кнопки интерфейса.

– Использование *BigDecimal* для обеспечения высокой точности математических вычислений.

– Поддержка двух конфигураций сборки (*Flavors*): «demo» (только базовые функции) и «full» (полный набор функций).

– Реализация математической логики вычислений с поддержкой приоритета операций и скобок без использования сторонних библиотек.

2 ВЫПОЛНЕНИЕ РАБОТЫ

2.1 Архитектура и состав приложения

Приложение реализовано на основе современного стека технологий разработки под *Android*. Архитектура следует принципам *MVVM* (*Model-View-ViewModel*), что обеспечивает четкое разделение логики вычислений и интерфейса.

На рисунке 2.1 представлена диаграмма классов приложения, отражающая основные сущности и их взаимодействия.

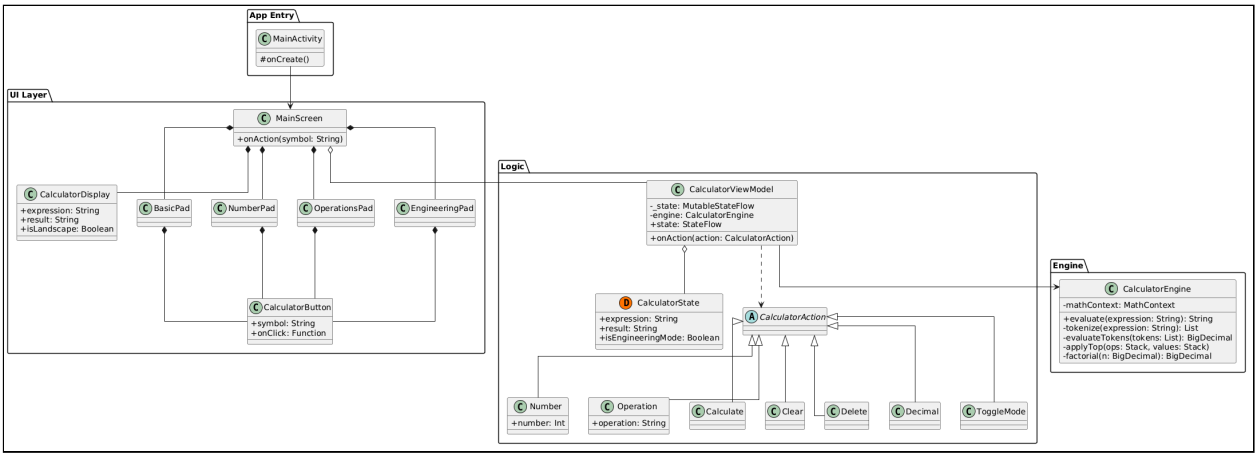

Рисунок 2.1 – Диаграмма классов приложения

Основные компоненты приложения:

– *Jetpack Compose*: используется для построения декларативного пользовательского интерфейса. Компоненты *BasicPad* и *EngineeringPad* описывают соответствующие наборы кнопок.

– *CalculatorViewModel*: управляет состоянием экрана, обрабатывает действия пользователя и взаимодействует с движком вычислений.

– *CalculatorEngine*: инкапсулирует математическую логику, выполняет токенизацию выражений и их вычисление.

2.2 Режимы работы и интерфейс

Приложение поддерживает два основных режима ввода: базовый и инженерный. Базовый режим включает стандартные арифметические операции и доступен во всех конфигурациях. Инженерный режим расширяет возможности калькулятора функциями возведения в степень, извлечения корня и работы со скобками.

Обработка ориентации устройства реализована в компоненте *MainScreen*. В ландшафтном режиме инженерная и базовая панели отображаются одновременно, предоставляя пользователю полный доступ ко всем функциям. В портретном режиме доступна кнопка переключения, которая поз-

воляет динамически изменять состав отображаемых кнопок. Для отображения истории ввода и результата используется компонент *CalculatorDisplay*.

На рисунках 2.2, 2.3 и 2.4 представлены интерфейсы приложения в различных состояниях и ориентациях.

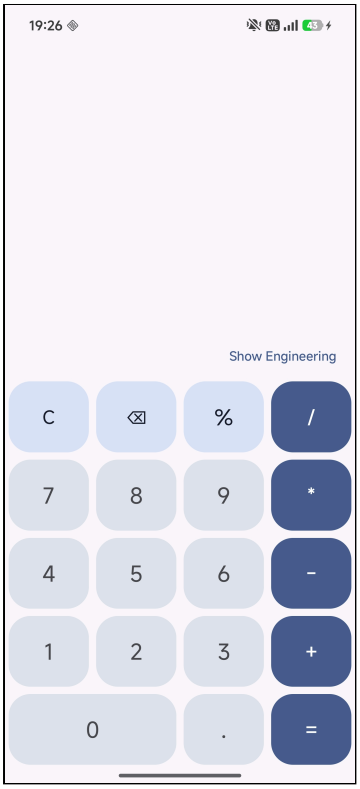

Рисунок 2.2 – Интерфейс калькулятора в портретной ориентации (базовый режим)

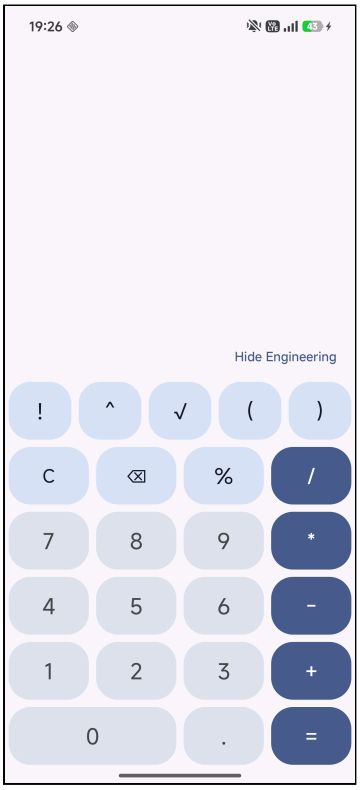

Рисунок 2.3 – Интерфейс калькулятора в портретной ориентации (инженерный режим)

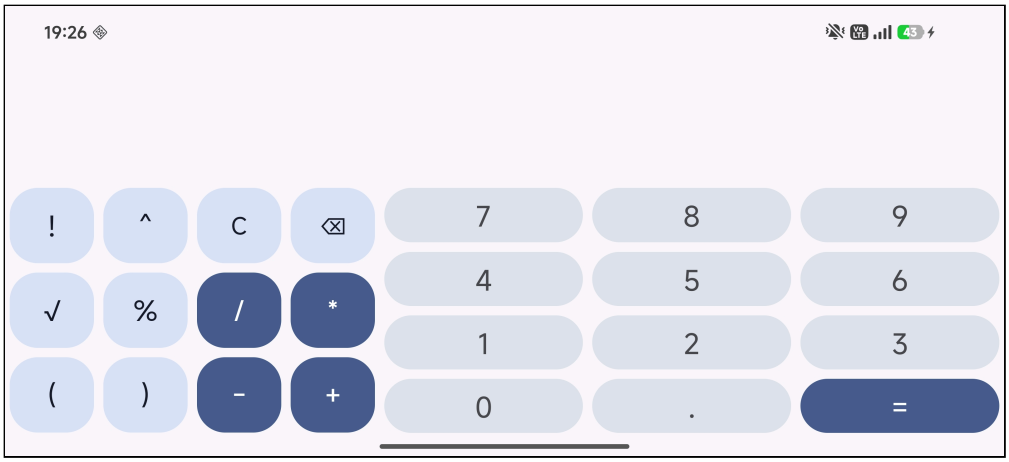

Рисунок 2.4 – Интерфейс калькулятора в ландшафтной ориентации

2.3 Математическая логика

Сердцем приложения является класс *CalculatorEngine*, реализующий логику разбора и вычисления математических выражений. Процесс вычисления разделен на три этапа:

1 Токенизация: входная строка разбивается на список токенов (числа, операторы, скобки). На этом этапе также обрабатываются унарные минусы.

2 Обработка и вычисление: токены анализируются с учетом приоритета операторов и их последовательности. Для хранения промежуточных значений и операторов используются стеки.

3 Обработка точности: все вычисления проводятся с использованием класса *BigDecimal*, что исключает ошибки округления, характерные для чисел с плавающей точкой (*Double*).

Реализация поддерживает обработку процентов, учитывая контекст операции (например, прибавление процента от числа).

2.4 Конфигурации сборки (Flavors)

В проекте настроены две конфигурации сборки (*product flavors*):

– *demo*: демонстрационная версия приложения, ограничивающая функциональность только базовыми операциями. Инженерный режим недоступен даже при смене ориентации или наличии соответствующих кнопок в коде (управление осуществляется через флаги *BuildConfig*).

– *full*: полнофункциональная версия приложения без ограничений.

Разделение реализовано в файле *build.gradle.kts*, что позволяет генерировать разные *APK* файлы с уникальными идентификаторами приложения.

ВЫВОД

В ходе выполнения лабораторной работы было разработано мобильное приложение «Калькулятор» на базе *Jetpack Compose*. Была успешно реализована математическая логика с поддержкой приоритета операций и использованием класса *BigDecimal* для обеспечения точности вычислений. Приложение демонстрирует продвинутую работу с интерфейсом: адаптацию под ориентацию экрана и динамическое переключение режимов работы. Настройка различных конфигураций сборки (*Flavors*) позволила разделить функциональность приложения на демонстрационную и полную версии, что является важным аспектом промышленной разработки мобильных приложений.

ПРИЛОЖЕНИЕ А
(обязательное)
Листинг программного кода

Листинг А.1 – Реализация алгоритма вычисления в CalculatorEngine

```
  private fun evaluateTokens(tokens: List<String>): BigDecimal {
      val values = Stack<BigDecimal>()
      val ops = Stack<String>()

      val precedence = mapOf(
          "+" to 1, "-" to 1,
          "*" to 2, "/" to 2,
          "^" to 3,
          "u-" to 4, "√" to 4
      )

      val isRightAssociative = setOf("^", "u-", "√")

      for (token in tokens) {
          when {
              isNumber(token) -> {
                  values.push(BigDecimal(token))
              }
              token == "(" -> {
                  ops.push(token)
              }
              token == ")" -> {
                  while (ops.isNotEmpty() && ops.peek() != "(") {
                      applyTop(ops, values)
                  }
                  if (ops.isNotEmpty()) ops.pop()
              }
              token == "!" -> {
                  if (values.isEmpty()) throw
IllegalArgumentException("Invalid syntax")
                  val a = values.pop()
                  values.push(factorial(a))
              }
              token == "%" -> {
                  if (values.isEmpty()) throw
IllegalArgumentException("Invalid syntax")
                  val a = values.pop()
                  if (ops.isNotEmpty() && (ops.peek() == "+" ||
ops.peek() == "-")) {
                      if (values.isEmpty()) throw
IllegalArgumentException("Invalid syntax")
                      val b = values.peek()

values.push(b.multiply(a).divide(BigDecimal(100), mathContext))
                  } else {
                      values.push(a.divide(BigDecimal(100),
mathContext))
                  }
              }
              token in precedence -> {
```

```
                    val p = precedence[token]!!
                    while (ops.isNotEmpty() && ops.peek() != "(") {
                        val top = ops.peek()
                        val topP = precedence[top] ?: 0

                        val condition = if (token in isRightAssociative)
{
                            topP > p
                        } else {
                            topP >= p
                        }

                        if (condition) {
                            applyTop(ops, values)
                        } else {
                            break
                        }
                    }
                    ops.push(token)
                }
                else -> throw IllegalArgumentException("Unknown token:
$token")
            }
        }

        while (ops.isNotEmpty()) {
            applyTop(ops, values)
        }

        if (values.size != 1) throw IllegalArgumentException("Invalid
syntax")
        return values.pop()
    }
```

Листинг А.2 – Логика переключения режимов и адаптивная верстка в MainScreen

```
  val isLandscape = configuration.orientation ==
Configuration.ORIENTATION_LANDSCAPE
  val isFullFlavor = BuildConfig.FLAVOR == "full"

  // ... внутри Column ...
  if (isFullFlavor) {
      Column(modifier = Modifier.fillMaxSize()) {
          if (!isLandscape) {
              // Кнопка переключения только в портретном режиме
              TextButton(
                  onClick =
{ viewModel.onAction(CalculatorAction.ToggleMode) }
              ) {
                  Text(if (state.isEngineeringMode) "Hide Engineering"
else "Show Engineering")
              }
          }

          if (isLandscape) {
```

```
                Row(modifier = Modifier.fillMaxWidth().weight(1f)) {
                    Row(modifier =
Modifier.weight(0.6f).fillMaxHeight()) {
                        EngineeringPad(onButtonClick = onAction,
isLandscape = true)
                        OperationsPad(onButtonClick = onAction)
                    }
                    NumberPad(onButtonClick = onAction)
                }
            } else {
                if (state.isEngineeringMode) {
                    EngineeringPad(onButtonClick = onAction, isLandscape
= false)
                    BasicPad(onButtonClick = onAction)
                } else {
                    BasicPad(onButtonClick = onAction)
                }
            }
        }
    } else {
        BasicPad(onButtonClick = onAction)
    }
```

Листинг А.3 – Конфигурация Flavors в build.gradle.kts

```
    flavorDimensions += "version"
    productFlavors {
        create("demo") {
            dimension = "version"
            applicationIdSuffix = ".demo"
            versionNameSuffix = "-demo"
        }
        create("full") {
            dimension = "version"
        }
    }
```